```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
#Import necessary libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
from sklearn.neighbors import KNeighborsRegressor
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error
from sklearn.cluster import KMeans
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (confusion_matrix, classification_report,
                             roc_auc_score, RocCurveDisplay)
#Set plot style
sns.set(style='whitegrid')

#Load the dataset
data = pd.read_csv('/content/VideoGames (1).csv')
data

#Display first few rows
data.head()
data.describe()
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6250 entries, 0 to 6249
Data columns (total 11 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Name             6250 non-null   object
 1   Platform         6250 non-null   object
 2   Year_of_Release  6250 non-null   int64
 3   Genre            6250 non-null   object
 4   NA_Sales         6229 non-null   float64
 5   EU_Sales         6236 non-null   float64
 6   JP_Sales         6233 non-null   float64
 7   Other_Sales      6236 non-null   float64
 8   Global_Sales     6232 non-null   float64
 9   Critic_Score     6231 non-null   float64
 10  User_Score       6235 non-null   float64
dtypes: float64(7), int64(1), object(3)
memory usage: 537.2+ KB
```

```
#Check for missing values
```

```
#Check for missing values
print(data.shape)
print('Missing Values:')
print(data.isnull().sum())

#Check data types
print('\nData Types:')
print(data.dtypes)

#Convert Critic_Score and User_Score to numeric, handling non-numeric values
#data['Critic_Score'] = pd.to_numeric(data['Critic_Score'], errors='coerce')
#data['User_Score'] = pd.to_numeric(data['User_Score'], errors='coerce')

#Impute missing scores with median
#imputer = SimpleImputer(strategy='mean')
#data[['Critic_Score', 'User_Score']] = imputer.fit_transform(data[['Critic_Score', 'User_Score']])
#data = data[data['Global_Sales'] <= 10].copy()
#Drop rows with missing sales data
data = data.dropna(subset=['NA_Sales', 'EU_Sales', 'JP_Sales', 'Other_Sales', 'Global_Sales','Critic_Score','User_Score'])

#Remove duplicates
data = data.drop_duplicates()

#Ensure sales are non-negative
data = data[(data['NA_Sales'] >= 0) & (data['EU_Sales'] >= 0) & (data['JP_Sales'] >= 0) &
            (data['Other_Sales'] >= 0) & (data['Global_Sales'] > 0)]

#Verify cleaning
print('\nAfter Cleaning - Missing Values:')
print(data.isnull().sum())
print('\nShape after cleaning:', data.shape)
```

```
(6250, 11)
Missing Values:
Name               0
Platform           0
Year_of_Release    0
Genre              0
NA_Sales          21
EU_Sales          14
JP_Sales          17
Other_Sales       14
Global_Sales      18
Critic_Score      19
User_Score        15
dtype: int64

Data Types:
Name               object
Platform           object
Year_of_Release     int64
Genre              object
NA_Sales          float64
EU_Sales          float64
JP_Sales          float64
Other_Sales       float64
Global_Sales      float64
Critic_Score      float64
User_Score        float64
dtype: object
```

```
After Cleaning - Missing Values:
Name               0
Platform           0
Year_of_Release    0
Genre              0
NA_Sales           0
EU_Sales           0
JP_Sales           0
Other_Sales        0
Global_Sales       0
Critic_Score       0
User_Score         0
dtype: int64

Shape after cleaning: (6142, 11)
```

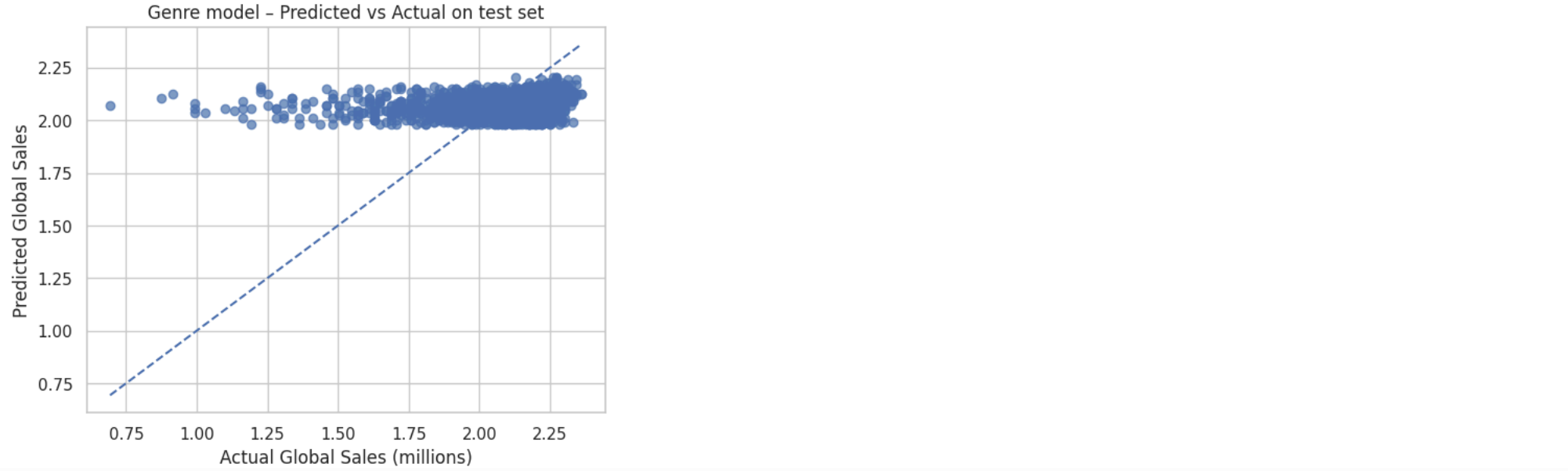
```
# Features and target
X = pd.get_dummies(data[['Critic_Score', 'User_Score', 'Genre']], columns=['Genre'], drop_first=True)
y = np.log1p(data['Global_Sales'])

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=4349)

model_genre = LinearRegression().fit(X_train, y_train)

# --- metrics ---
r2_train = model_genre.score(X_train, y_train)
r2_test  = model_genre.score(X_test,  y_test)
y_pred = model_genre.predict(X_test)
n_train, p = X_train.shape
adj_r2_train = 1 - (1 - r2_train) * (n_train - 1) / (n_train - p - 1)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print(f"Train R²:        {r2_train:.4f}")
print(f"Train adj R²:    {adj_r2_train:.4f}")
print(f"Test  R²:        {r2_test:.4f}")
print(f"Test  RMSE:      {rmse:.4f}  (millions)")


# Plot predicted vs actual sales
plt.scatter(y_test, y_pred, alpha=0.7)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()], linestyle='--')
plt.xlabel('Actual Global Sales (millions)')
plt.ylabel('Predicted Global Sales')
plt.title('Genre model – Predicted vs Actual on test set')
plt.grid(True)
plt.show()
```

```
Train R²:        0.1871
Train adj R²:    0.1846
Test  R²:        0.1730
Test  RMSE:      0.4349  (millions)
```


```
# Features and target
X = pd.get_dummies(data[['Year_of_Release']], drop_first=True)
y = np.log1p(data['User_Score'])

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=502)

model_genre = LinearRegression().fit(X_train, y_train)

# --- metrics ---
r2_train = model_genre.score(X_train, y_train)
r2_test  = model_genre.score(X_test,  y_test)
y_pred = model_genre.predict(X_test)
n_train, p = X_train.shape
adj_r2_train = 1 - (1 - r2_train) * (n_train - 1) / (n_train - p - 1)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print(f"Train R²:        {r2_train:.4f}")
print(f"Train adj R²:    {adj_r2_train:.4f}")
print(f"Test  R²:        {r2_test:.4f}")
print(f"Test  RMSE:      {rmse:.4f}  (millions)")


# Plot predicted vs actual sales
plt.scatter(y_test, y_pred, alpha=0.7)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()], linestyle='--')
plt.xlabel('Actual Global Sales (millions)')
```

```
plt.ylabel('Predicted Global Sales')
plt.title('Genre model – Predicted vs Actual on test set')
plt.grid(True)
plt.show()
```

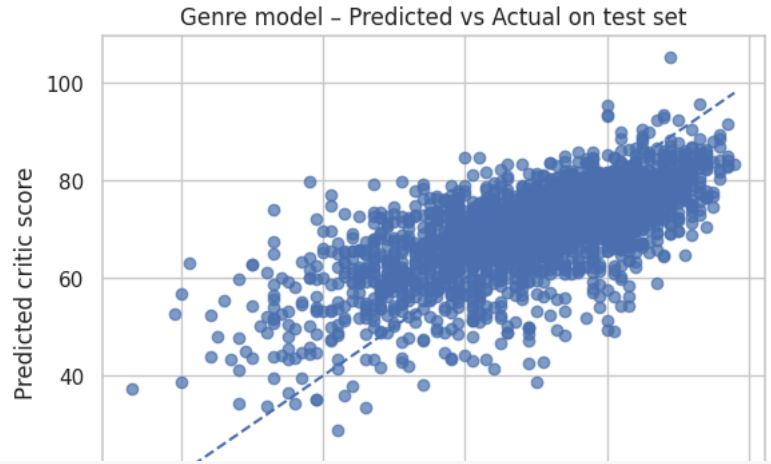
```
Train R²:        0.0470
Train adj R²:    0.0467
Test  R²:        0.0500
Test  RMSE:      0.2060  (millions)
```




```
# Features and target
X = pd.get_dummies(data[['User_Score','Global_Sales','Platform', 'Genre','Year_of_Release']], drop_first=True)
y = (data['Critic_Score'])

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=123)

model_genre = LinearRegression().fit(X_train, y_train)

# --- metrics ---
r2_train = model_genre.score(X_train, y_train)
r2_test  = model_genre.score(X_test,  y_test)
y_pred = model_genre.predict(X_test)
n_train, p = X_train.shape
adj_r2_train = 1 - (1 - r2_train) * (n_train - 1) / (n_train - p - 1)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print(f"Train R²:        {r2_train:.4f}")
print(f"Train adj R²:    {adj_r2_train:.4f}")
print(f"Test  R²:        {r2_test:.4f}")
print(f"Test  RMSE:      {rmse:.4f}  (Rating)")

intercept  = model_genre.intercept_
```

```
coeffs     = pd.Series(model_genre.coef_, index=X.columns)

# Nice, readable list
print("\nLinear-model coefficients:")
print(f"Intercept = {intercept:.4f}")
for name, coef in coeffs.sort_values(key=abs, ascending=False).items():
    print(f"{name:<30} {coef:+.4f}")
# Plot predicted vs actual sales
plt.scatter(y_test, y_pred, alpha=0.7)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()], linestyle='--')
plt.xlabel('Actual critic score')
plt.ylabel('Predicted critic score')
plt.title('Genre model – Predicted vs Actual on test set')
plt.grid(True)
plt.show()
```

```
Train R²:        0.4621
Train adj R²:    0.4576
Test  R²:        0.4568
Test  RMSE:      10.2777  (Rating)

Linear-model coefficients:
Intercept = -587.8218
Platform_DC                    +16.6280
Platform_PC                    +7.9414
Genre_Sports                   +7.2267
Genre_Puzzle                   +6.1224
User_Score                     +5.8016
Platform_Wii                   -4.8982
Platform_XOne                  +4.5734
Platform_PS                    +3.3952
Genre_Misc                     +3.1790
Platform_PS4                   +2.7830
Platform_PS3                   +2.7582
Genre_Strategy                 +2.5300
Genre_Racing                   +2.4251
Genre_Shooter                  +2.2082
Genre_Role-Playing             +2.1980
Genre_Simulation               +2.1551
Genre_Platform                 +1.8192
Platform_DS                    -1.7208
Platform_GBA                   -1.7067
Platform_PS2                   -1.6161
Platform_XB                    +1.5100
Platform_PSV                   -1.3587
Genre_Fighting                 +1.2262
Platform_X360                  +1.1398
Global_Sales                   +1.0642
Platform_WiiU                  +1.0597
Platform_PSP                   -1.0582
Platform_GC                    +0.8660
Genre_Adventure                -0.8464
Year_of_Release                +0.3050
```




```
gp_stats = (data.groupby(['Genre'])
              .agg(avg_NA      = ('NA_Sales',     'mean'),
                   avg_EU      = ('EU_Sales',     'mean'),
                   avg_JP      = ('JP_Sales',     'mean'),
```

```
                    avg_Other   = ('Other_Sales',  'mean'),
                    avg_critic  = ('Critic_Score', 'mean'),
                    avg_user    = ('User_Score',   'mean'),
                    n_games     = ('Genre',        'size'))   # extra info
                .reset_index())

# 2. Feature matrix (numeric only for K-means) ------------------------------

X = gp_stats[[ 'avg_NA', 'avg_EU', 'avg_JP', 'avg_Other',
               'avg_critic', 'avg_user']]

# 3. Scale so each metric contributes equally -------------------------------

scaler   = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 4. Run K-means ------------------------------------------------------------

k       = 4                   # change 2…6 if you like
kmeans  = KMeans(n_clusters=k, random_state=0, n_init='auto')
labels  = kmeans.fit_predict(X_scaled)
gp_stats['cluster'] = labels

print("Silhouette score:", round(silhouette_score(X_scaled, labels), 3))

# 5. Detailed cluster view --------------------------------------------------

print("\nGenre × Platform clusters:")
for c in sorted(gp_stats['cluster'].unique()):
    print(f"\n—— Cluster {c} ——")
    display(gp_stats.loc[gp_stats['cluster'] == c]
                    .sort_values('avg_user', ascending=False))

# 6. Quick numeric summary per cluster --------------------------------------

summary = (gp_stats.groupby('cluster')
                        .agg(pairs        = ('cluster',    'size'),
                             mean_NA      = ('avg_NA',     'mean'),
                             mean_EU      = ('avg_EU',     'mean'),
                             mean_JP      = ('avg_JP',     'mean'),
                             mean_Other   = ('avg_Other',  'mean'),
                             mean_cscore  = ('avg_critic', 'mean'),
                             mean_uscore  = ('avg_user',   'mean'))
                        .round(2))
print("\nCluster-level means:\n", summary)
```

```
Silhouette score: 0.236

Genre × Platform clusters:

── Cluster 0 ──
```

	Genre	avg_NA	avg_EU	avg_JP	avg_Other	avg_critic	avg_user	n_games	cluster
6	Racing	0.411550	0.300523	0.054535	0.106143	69.153101	7.093411	516	0
8	Shooter	0.526008	0.309677	0.022351	0.104354	70.789406	7.087597	774	0
10	Sports	0.497699	0.272418	0.039042	0.103773	74.037383	7.058879	856	0
0	Action	0.366592	0.240971	0.046959	0.092376	67.380177	7.056687	1473	0
3	Misc	0.601647	0.323671	0.093555	0.108584	66.942197	6.789306	346	0

```
── Cluster 1 ──
```

	Genre	avg_NA	avg_EU	avg_JP	avg_Other	avg_critic	avg_user	n_games	cluster
4	Platform	0.482715	0.270665	0.109169	0.082881	69.559557	7.303324	361	1
2	Fighting	0.358653	0.161138	0.075539	0.067275	68.940120	7.249401	334	1
5	Puzzle	0.297685	0.218704	0.137130	0.057130	71.027778	7.230556	108	1
9	Simulation	0.320746	0.232463	0.098172	0.059142	69.839552	7.150373	268	1

```
── Cluster 2 ──
```

	Genre	avg_NA	avg_EU	avg_JP	avg_Other	avg_critic	avg_user	n_games	cluster
7	Role-Playing	0.31206	0.173043	0.183233	0.058716	72.527734	7.580507	631	2

```
── Cluster 3 ──
```

	Genre	avg_NA	avg_EU	avg_JP	avg_Other	avg_critic	avg_user	n_games	cluster
11	Strategy	0.125732	0.092971	0.017448	0.023766	72.958159	7.255230	239	3
1	Adventure	0.151737	0.098390	0.031907	0.031483	66.016949	7.085593	236	3

```
Cluster-level means:
          pairs  mean_NA  mean_EU  mean_JP  mean_Other  mean_cscore  \
cluster
0             5     0.48     0.29     0.05        0.10        69.66
1             4     0.36     0.22     0.11        0.07        69.84
2             1     0.31     0.17     0.18        0.06        72.53
3             2     0.14     0.10     0.02        0.03        69.49

         mean_uscore
cluster
0               7.02
1               7.23
2               7.58
3               7.17
```

```
gp_stats = (data.groupby(['Genre', 'Platform'])
              .agg(
                   avg_NA      = ('NA_Sales',     'mean'),
                   avg_EU      = ('EU_Sales',     'mean'),
                   avg_JP      = ('JP_Sales',     'mean'),
```

```
                    avg_Other   = ('Other_Sales',  'mean'),
                    avg_critic  = ('Critic_Score', 'mean'),
                    avg_user    = ('User_Score',   'mean'),
                    n_games     = ('Genre',        'size'))   # games in pair
                .reset_index())

# 2. Feature matrix ----------------------------------------------------------
X = gp_stats[[ 'avg_NA', 'avg_EU', 'avg_JP', 'avg_Other',
               'avg_critic', 'avg_user']]

# 3. Standardise numeric features --------------------------------------------
scaler   = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 4. K-means -----------------------------------------------------------------
k       = 4
kmeans  = KMeans(n_clusters=k, random_state=0, n_init='auto')
labels  = kmeans.fit_predict(X_scaled)
gp_stats['cluster'] = labels

print("Silhouette score:", round(silhouette_score(X_scaled, labels), 3))

# 5. For each cluster → show one row per Genre with its *dominant* platform
print("\nDominant platform per Genre (within each cluster):")
for c in sorted(gp_stats['cluster'].unique()):
    print(f"\n── Cluster {c} ──")
    clust = gp_stats.loc[gp_stats['cluster'] == c]

    # pick the platform with the most games for each genre
    dom = (clust.sort_values('n_games', ascending=False)
                .groupby('Genre', as_index=False)
                .first())   # keep the first row per genre (dom platform)

    display(dom[['Genre', 'Platform', 'n_games','avg_NA', 'avg_EU', 'avg_JP', 'avg_Other', 'avg_critic', 'avg_user']]
            .sort_values('avg_user', ascending=False))

# 6. Quick numeric summary per cluster ---------------------------------------
summary = (gp_stats.groupby('cluster')
                        .agg(pairs        = ('cluster',    'size'),
                             mean_NA      = ('avg_NA',     'mean'),
                             mean_EU      = ('avg_EU',     'mean'),
                             mean_JP      = ('avg_JP',     'mean'),
                             mean_Other   = ('avg_Other',  'mean'),
                             mean_cscore  = ('avg_critic', 'mean'),
                             mean_uscore  = ('avg_user',   'mean'))
                        .round(2))
print("\nCluster-level means:\n", summary)
```

```
Silhouette score: 0.285

Dominant platform per Genre (within each cluster):

── Cluster 0 ──
```

	Genre	Platform	n_games	avg_NA	avg_EU	avg_JP	avg_Other	avg_critic	avg_user
2	Fighting	PS2	67	0.439403	0.210000	0.102687	0.120000	70.044776	7.961194
10	Sports	PS2	177	0.553107	0.249379	0.046384	0.166836	74.988701	7.687571
0	Action	PS2	215	0.466186	0.264651	0.085070	0.180140	66.437209	7.640000
6	Racing	PS2	113	0.532389	0.308496	0.034336	0.186018	68.442478	7.638938
5	Puzzle	DS	50	0.364000	0.314400	0.240400	0.082000	70.200000	7.170000
4	Platform	Wii	33	1.193030	0.596970	0.289697	0.176364	68.363636	6.960606
7	Role-Playing	X360	51	0.770980	0.307647	0.040980	0.106471	70.803922	6.937255
1	Adventure	PS3	18	0.357222	0.338333	0.037222	0.134444	68.333333	6.838889
8	Shooter	X360	139	1.068705	0.452446	0.016763	0.152014	69.482014	6.807194
3	Misc	Wii	65	1.138000	0.694308	0.138615	0.201538	64.215385	6.660000
9	Simulation	DS	32	0.500000	0.338750	0.281562	0.099687	64.656250	6.428125

```
── Cluster 1 ──
```

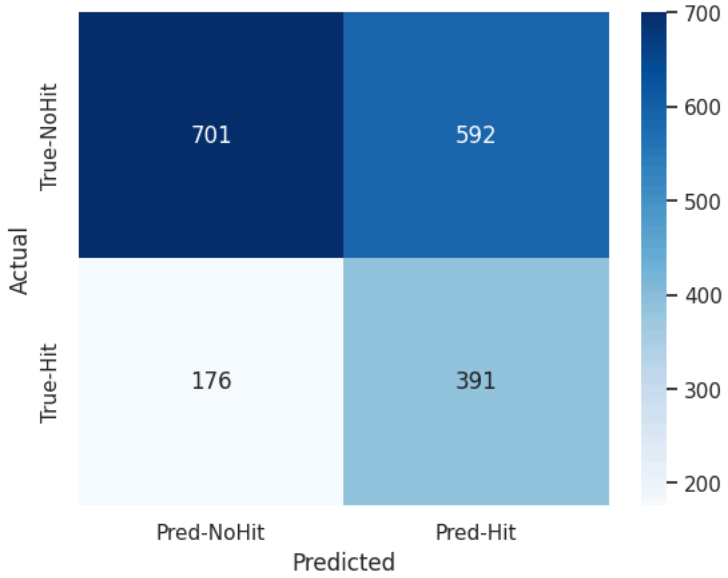
	Genre	Platform	n_games	avg_NA	avg_EU	avg_JP	avg_Other	avg_critic	avg_user
8	Shooter	PS2	116	0.407672	0.252672	0.015517	0.090776	66.818966	7.567241
4	Platform	PS2	63	0.390000	0.245238	0.018095	0.105397	66.031746	7.460317
5	Puzzle	Wii	13	0.259231	0.171538	0.060000	0.048462	69.692308	7.300000
6	Racing	XB	70	0.208714	0.082857	0.001714	0.010286	68.914286	7.262857
7	Role-Playing	PS3	71	0.344366	0.217746	0.163803	0.097887	70.746479	7.188732
11	Strategy	PC	107	0.064860	0.120093	0.000000	0.022150	75.065421	7.144860
1	Adventure	PS2	34	0.096176	0.075294	0.029118	0.029412	60.941176	6.911765
0	Action	X360	205	0.538878	0.284683	0.010878	0.083122	66.590244	6.851707
9	Simulation	PC	72	0.210417	0.290694	0.000000	0.042500	73.375000	6.843056
2	Fighting	X360	46	0.424130	0.151087	0.012391	0.055652	68.239130	6.615217
10	Sports	X360	128	0.519844	0.184922	0.001719	0.063906	70.664062	6.528906
3	Misc	PS3	36	0.342500	0.178889	0.011667	0.083333	71.944444	6.475000

```
── Cluster 2 ──
```

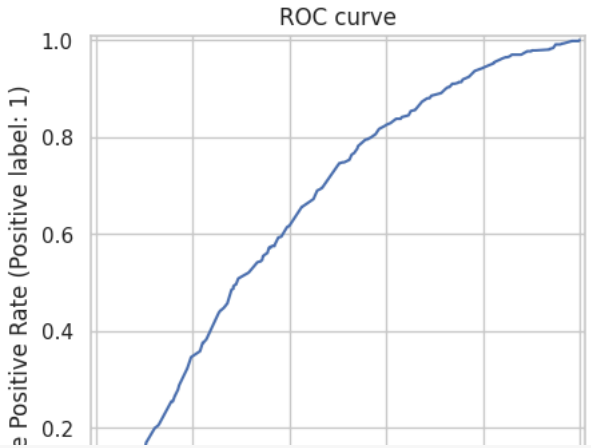
	Genre	Platform	n_games	avg_NA	avg_EU	avg_JP	avg_Other	avg_critic	avg_user
0	Racing	WiiU	1	3.15	2.15	1.28	0.51	88.0	9.1

```
MILESTONE = 0.6          # hit threshold (millions of copies)

# 1. Load & minimal clean ------------------------------------------------
df = pd.read_csv('/content/VideoGames (1).csv')
```

```
df['User_Score'] = pd.to_numeric(df['User_Score'], errors='coerce')
df = df[['Genre','Platform','Critic_Score','User_Score','Global_Sales']].dropna()

# 2. Binary label ------------------------------------------------------
df['hit'] = (df['Global_Sales'] >= MILESTONE).astype(int)

# 3. Historical genre-average scores -----------------------------------
g_means = (df.groupby('Genre')
             .agg(genre_avg_critic=('Critic_Score', 'mean'),
                  genre_avg_user  =('User_Score',   'mean')))
df = df.join(g_means, on='Genre')

# 4. Feature matrix (one-hot cats) -------------------------------------
X = pd.get_dummies(
        df[['Platform','Genre','genre_avg_critic','genre_avg_user']],
        columns=['Platform','Genre'],
        drop_first=True
    )
y = df['hit']

# Identify the numeric columns for scaling
num_cols = ['genre_avg_critic', 'genre_avg_user']

# 5. Train–test split --------------------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.30, random_state=42, stratify=y)

# 6. Standardise numeric columns (fit on train only) -------------------
scaler = StandardScaler()
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols]  = scaler.transform(X_test[num_cols])

# 7. Fit logistic regression -------------------------------------------
logreg = LogisticRegression(max_iter=500, class_weight='balanced')
logreg.fit(X_train, y_train)

# 8. Evaluation --------------------------------------------------------
y_proba = logreg.predict_proba(X_test)[:, 1]
threshold = 0.5         # try 0.30, 0.20, etc. in a loop or ROC curve
y_pred = (y_proba >= threshold).astype(int)
# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Pred-NoHit','Pred-Hit'],
            yticklabels=['True-NoHit','True-Hit'])
plt.ylabel('Actual'); plt.xlabel('Predicted'); plt.show()

# Classification report
print("\nClassification report:")
print(classification_report(y_test, y_pred, digits=3))

# ROC–AUC
roc_auc = roc_auc_score(y_test, y_proba)
print(f"ROC-AUC: {roc_auc:.3f}")
RocCurveDisplay.from_predictions(y_test, y_proba)
plt.title('ROC curve'); plt.show()
```




```
Classification report:
              precision    recall  f1-score   support

           0      0.799     0.542     0.646      1293
           1      0.398     0.690     0.505       567

    accuracy                          0.587      1860
   macro avg      0.599     0.616     0.575      1860
weighted avg      0.677     0.587     0.603      1860

ROC-AUC: 0.649
```




```
df = pd.read_csv('/content/VideoGames (1).csv')
df['User_Score'] = pd.to_numeric(df['User_Score'], errors='coerce')

cols = ['Name','Genre','Platform','Critic_Score','User_Score',
```

```
        'Year_of_Release','Global_Sales']
df   = df[cols].dropna().reset_index(drop=True)

# 2. Feature matrix ------------------------------------------------------
X = pd.get_dummies(
        df[['Genre','Platform','Critic_Score','User_Score']],
        columns=['Genre','Platform'],
        drop_first=True)

y = np.log1p(df['Global_Sales'])          # work on log-sales for stability

# Identify numeric columns to scale
num_cols = ['Critic_Score','User_Score']
scaler   = StandardScaler()

# 3. Train–test split + scale --------------------------------------------
X_train, X_test, y_train, y_test, idx_train, idx_test = train_test_split(
        X, y, df.index, test_size=0.30, random_state=42)

X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols]  = scaler.transform(X_test[num_cols])

# 4. k-NN regressor ------------------------------------------------------
knn = KNeighborsRegressor(n_neighbors=60, weights='distance')
knn.fit(X_train, y_train)

# 5. Evaluation ----------------------------------------------------------
y_pred  = knn.predict(X_test)
mae_log = mean_absolute_error(y_test, y_pred)
r2      = r2_score(y_test, y_pred)

print(f"Test MAE (log units): {mae_log:.3f}")
print(f"Test R²:              {r2:.3f}")
print(f"≈ multiplicative MAE: {np.expm1(mae_log):.2%}")

# 6. Demo: predict a hypothetical new game -------------------------------
new_game = pd.DataFrame({
    'Genre'        : ['Action'],
    'Platform'     : ['PS4'],
    'Critic_Score' : [85],
    'User_Score'   : [8.2]
})

newX = pd.get_dummies(new_game, columns=['Genre','Platform'], drop_first=True)
# align columns with training set (missing dummies → 0)
newX = newX.reindex(columns=X.columns, fill_value=0)

# scale numeric cols
newX[num_cols] = scaler.transform(newX[num_cols])

pred_log = knn.predict(newX)[0]
pred_lin = np.expm1(pred_log)
print(f"\nPredicted global sales for the new game ≈ {pred_lin:.2f} million units")

# 7. Show the 5 nearest neighbours ---------------------------------------
dists, neigh_idx = knn.kneighbors(newX, n_neighbors=5, return_distance=True)
print("\nNearest historical titles:")
```

```
for rank, (row_i, dist) in enumerate(zip(idx_train[neigh_idx[0]], dists[0]), 1):
    row = df.loc[row_i]
    print(f"{rank}. {row['Name']:<35}  "
          f"{row['Global_Sales']:.2f} M  "
          f"(Critic {row['Critic_Score']}, User {row['User_Score']})  "
          f"dist={dist:.2f}")
```

```
Test MAE (log units): 0.260
Test R²:              0.345
≈ multiplicative MAE: 29.76%

Predicted global sales for the new game ≈ 1.08 million units

Nearest historical titles:
1. Luigi's Mansion: Dark Moon           4.59 M  (Critic 86.0, User 8.4)  dist=0.15
2. Resident Evil: Revelations           0.88 M  (Critic 82.0, User 8.5)  dist=0.30
3. Kid Icarus: Uprising                 1.27 M  (Critic 83.0, User 8.7)  dist=0.37
4. Kirby: Planet Robobot                0.93 M  (Critic 81.0, User 8.7)  dist=0.45
5. Art Academy: Lessons for Everyone    0.46 M  (Critic 81.0, User 7.6)  dist=0.50
```

```
# --------- K-means on (Genre × Decade) profiles ---------------------------
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# 0.  Add a decade column ----------------------------------------------------
data = data.copy()
data['Decade'] = (data['Year_of_Release'] // 10) * 10    # e.g. 1993 → 1990

# 1.  Aggregate one row per (Genre, Decade) ----------------------------------
gp_stats = (data.groupby(['Genre', 'Decade'])
              .agg(avg_NA     = ('NA_Sales',     'mean'),
                   avg_EU     = ('EU_Sales',     'mean'),
                   avg_JP     = ('JP_Sales',     'mean'),
                   avg_Other  = ('Other_Sales',  'mean'),
                   avg_critic = ('Critic_Score', 'mean'),
                   avg_user   = ('User_Score',   'mean'),
                   n_games    = ('Genre',        'size'))
              .reset_index())

# 2.  Feature matrix ---------------------------------------------------------
X = gp_stats[['avg_NA','avg_EU','avg_JP','avg_Other',
              'avg_critic','avg_user']]

# 3.  Standardise ------------------------------------------------------------
scaler   = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 4.  K-means ----------------------------------------------------------------
k       = 2                  # change to 2…6 and re-run if you like
kmeans  = KMeans(n_clusters=k, random_state=0, n_init='auto')
labels  = kmeans.fit_predict(X_scaled)
gp_stats['cluster'] = labels

print("Silhouette score:", round(silhouette_score(X_scaled, labels), 3))
```

```
# 5.  Detailed view ------------------------------------------------------
print("\nGenre × Decade clusters:")
for c in sorted(gp_stats['cluster'].unique()):
    print(f"\n── Cluster {c} ──")
    display(gp_stats.loc[gp_stats['cluster'] == c]
                    .sort_values('avg_user', ascending=False)
                    .reset_index(drop=True))

# 6.  Numeric summary ----------------------------------------------------
summary = (gp_stats.groupby('cluster')
                       .agg(pairs       = ('cluster',    'size'),
                            mean_NA     = ('avg_NA',     'mean'),
                            mean_EU     = ('avg_EU',     'mean'),
                            mean_JP     = ('avg_JP',     'mean'),
                            mean_Other  = ('avg_Other',  'mean'),
                            mean_cscore = ('avg_critic', 'mean'),
                            mean_uscore = ('avg_user',   'mean'))
                       .round(2))
print("\nCluster-level means:\n", summary)
```

```
Silhouette score: 0.517

Genre × Decade clusters:

── Cluster 0 ──
```

	Genre	Decade	avg_NA	avg_EU	avg_JP	avg_Other	avg_critic	avg_user	n_games	cluster
0	Strategy	1990	0.016000	0.038000	0.000000	0.004000	81.600000	8.320000	5	0
1	Simulation	1990	0.130000	0.087500	0.455000	0.040000	85.500000	8.175000	4	0
2	Puzzle	1990	0.075000	0.050000	0.000000	0.010000	81.000000	7.850000	2	0
3	Role-Playing	2000	0.304128	0.142026	0.191179	0.050923	71.951282	7.747949	390	0
4	Adventure	1990	0.270000	0.125000	0.250000	0.025000	68.500000	7.550000	2	0
5	Simulation	2000	0.360199	0.233134	0.082687	0.062090	71.089552	7.483582	201	0
6	Strategy	2000	0.117384	0.074302	0.021919	0.018547	72.523256	7.412209	172	0
7	Adventure	2010	0.130375	0.123750	0.023375	0.038500	71.400000	7.393750	80	0
8	Fighting	2000	0.396712	0.157534	0.070731	0.072420	68.392694	7.386758	219	0
9	Shooter	2000	0.432133	0.211781	0.011644	0.072916	70.050881	7.380431	511	0
10	Sports	2000	0.508917	0.243280	0.043885	0.099013	74.675159	7.370223	628	0
11	Platform	2000	0.468815	0.245000	0.088667	0.080333	67.818519	7.326667	270	0
12	Racing	2000	0.427918	0.271465	0.042725	0.109332	68.272494	7.286118	389	0
13	Puzzle	2000	0.311932	0.239091	0.153977	0.062841	70.238636	7.255682	88	0
14	Action	2000	0.368788	0.196049	0.042005	0.089009	65.814685	7.229487	858	0
15	Role-Playing	2010	0.302838	0.204820	0.125000	0.068829	72.680180	7.209009	222	0
16	Platform	2010	0.486824	0.330471	0.159059	0.089882	73.929412	7.124706	85	0
17	Puzzle	2010	0.252778	0.137778	0.070000	0.034444	73.777778	7.038889	18	0
18	Adventure	2000	0.161299	0.084870	0.033506	0.027922	63.188312	6.919481	154	0
19	Fighting	2010	0.246075	0.136822	0.051402	0.055514	69.299065	6.909346	107	0
20	Misc	2000	0.604085	0.335830	0.094213	0.116809	65.970213	6.828511	235	0
21	Action	2010	0.341838	0.287550	0.047185	0.094636	69.278146	6.783775	604	0
22	Strategy	2010	0.157742	0.149194	0.006452	0.039839	73.467742	6.733871	62	0
23	Misc	2010	0.604679	0.301651	0.078807	0.092385	68.816514	6.679817	109	0
24	Shooter	2010	0.697799	0.503089	0.043475	0.167413	72.065637	6.489189	259	0
25	Racing	2010	0.272353	0.327647	0.050924	0.089916	70.705882	6.351261	119	0
26	Sports	2010	0.455796	0.349690	0.022699	0.116814	72.097345	6.176991	226	0
27	Simulation	2010	0.213770	0.246557	0.129016	0.052295	64.967213	6.088525	61	0

```
#我们要用的11111111111111
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
data=pd.read_csv("/content/VideoGames (1).csv")

#Data Cleaning and Preparation
#Convert Critic_Score and User_Score to numeric, handle non-numeric values
data = data.copy()
data['Critic_Score'] = pd.to_numeric(data['Critic_Score'], errors='coerce')
data['User_Score'] = pd.to_numeric(data['User_Score'], errors='coerce')

#Add Decade column (e.g., 1993 → 1990)
#1996–2005 → 2000
#2006–2015 → 2010
#2016–2025 → 2020
data['Decade'] = (data['Year_of_Release'] // 10) * 10

#Aggregate one row per (Genre, Decade)
gp_stats = (data.groupby(['Genre', 'Decade'])
              .agg(avg_NA     = ('NA_Sales',     'mean'),
                   avg_EU     = ('EU_Sales',     'mean'),
                   avg_JP     = ('JP_Sales',     'mean'),
                   avg_Other  = ('Other_Sales',  'mean'),
                   avg_critic = ('Critic_Score', 'mean'),
                   avg_user   = ('User_Score',   'mean'),
                   n_games    = ('Genre',        'count'))
              .reset_index())

#Feature matrix for clustering
X = gp_stats[['avg_NA', 'avg_EU', 'avg_JP', 'avg_Other', 'avg_critic', 'avg_user']]

#Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

#K-means clustering
k = 2  # Adjusted to 3 clusters for better separation; can experiment with 2–6
kmeans = KMeans(n_clusters=k, random_state=0, n_init=10)
labels = kmeans.fit_predict(X_scaled)
gp_stats['cluster'] = labels

#Silhouette score
print("Silhouette score:", round(silhouette_score(X_scaled, labels), 3))

#Detailed view of clusters
print("\nGenre × Decade clusters:")
for c in sorted(gp_stats['cluster'].unique()):
    print(f"\n—— Cluster {c} ——")
    display(gp_stats.loc[gp_stats['cluster'] == c]
                    .sort_values('avg_user', ascending=False)
                    .reset_index(drop=True))

#Numeric summary of clusters
summary = (gp_stats.groupby('cluster')
                   .agg(pairs       = ('cluster',    'size'),
                        mean_NA     = ('avg_NA',     'mean'),
                        mean_EU     = ('avg_EU',     'mean'),
                        mean_JP     = ('avg_JP',     'mean'),
```

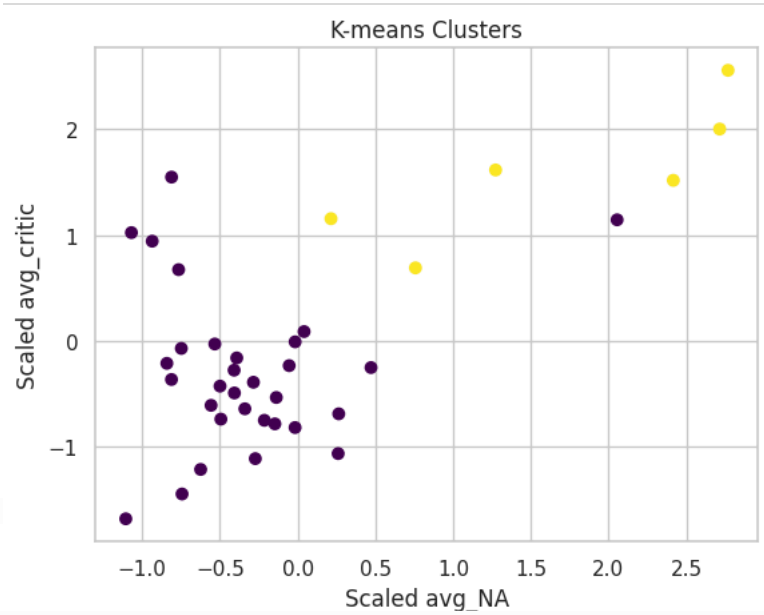
```
                        mean_Other  = ('avg_Other',  'mean'),
                        mean_cscore = ('avg_critic', 'mean'),
                        mean_uscore = ('avg_user',   'mean'))
                    .round(2))
print("\nCluster-level means:\n", summary)
```

```
Silhouette score: 0.536

Genre × Decade clusters:

── Cluster 0 ──
```

	Genre	Decade	avg_NA	avg_EU	avg_JP	avg_Other	avg_critic	avg_user	n_games	cluster
0	Shooter	1990	1.395000	0.292500	0.022500	0.037500	82.500000	8.425000	4	0
1	Strategy	1990	0.016000	0.038000	0.000000	0.004000	81.600000	8.320000	5	0
2	Simulation	1990	0.130000	0.087500	0.455000	0.040000	85.500000	8.175000	4	0
3	Misc	1990	0.150000	0.095000	0.820000	0.025000	79.000000	8.150000	2	0
4	Puzzle	1990	0.075000	0.050000	0.000000	0.010000	81.000000	7.850000	2	0
5	Role-Playing	2000	0.307582	0.142343	0.188759	0.050707	71.936869	7.746582	397	0
6	Adventure	1990	0.270000	0.125000	0.250000	0.025000	68.500000	7.550000	2	0
7	Simulation	2000	0.362426	0.233134	0.082277	0.062129	71.089552	7.476733	202	0
8	Strategy	2000	0.116609	0.073506	0.021543	0.018391	72.419540	7.410857	175	0
9	Adventure	2010	0.129753	0.122469	0.023086	0.038500	71.283951	7.393827	81	0
10	Fighting	2000	0.393423	0.157568	0.070000	0.072072	68.417040	7.390135	223	0
11	Shooter	2000	0.427399	0.209615	0.011525	0.072293	70.021277	7.377606	520	0
12	Sports	2000	0.506246	0.247928	0.044403	0.099480	74.649371	7.375667	637	0
13	Platform	2000	0.480662	0.249596	0.090879	0.080294	67.908425	7.323810	273	0
14	Racing	2000	0.423342	0.267909	0.041914	0.107638	68.168342	7.272796	399	0
15	Puzzle	2000	0.308427	0.239091	0.153146	0.062135	70.337079	7.260674	89	0
16	Action	2000	0.367759	0.196410	0.041972	0.089402	65.726437	7.232221	871	0
17	Role-Playing	2010	0.315044	0.205771	0.125833	0.071096	72.798246	7.222026	228	0
18	Platform	2010	0.480805	0.332299	0.155517	0.090805	73.929412	7.154023	87	0
19	Puzzle	2010	0.252778	0.137778	0.070000	0.034444	73.777778	7.038889	18	0
20	Fighting	2010	0.242162	0.134286	0.052432	0.054107	69.468468	6.945946	112	0
21	Adventure	2000	0.159744	0.083924	0.032866	0.027658	63.242038	6.943038	158	0
22	Misc	2000	0.602983	0.335527	0.093025	0.116807	66.080169	6.830802	238	0
23	Action	2010	0.338114	0.284441	0.046331	0.093528	69.225284	6.779383	618	0
24	Strategy	2010	0.157742	0.149194	0.006452	0.039839	73.467742	6.733871	62	0
25	Misc	2010	0.604679	0.313909	0.081091	0.094091	68.872727	6.691818	110	0
26	Shooter	2010	0.696312	0.517072	0.043118	0.170000	72.122137	6.482890	264	0
27	Racing	2010	0.267623	0.324711	0.050656	0.088279	70.818182	6.333058	122	0

```
#我们要用的22222222222
import matplotlib.pyplot as plt
plt.scatter(X_scaled[:, 0], X_scaled[:, 4], c=labels, cmap='viridis')
plt.xlabel('Scaled avg_NA')
```

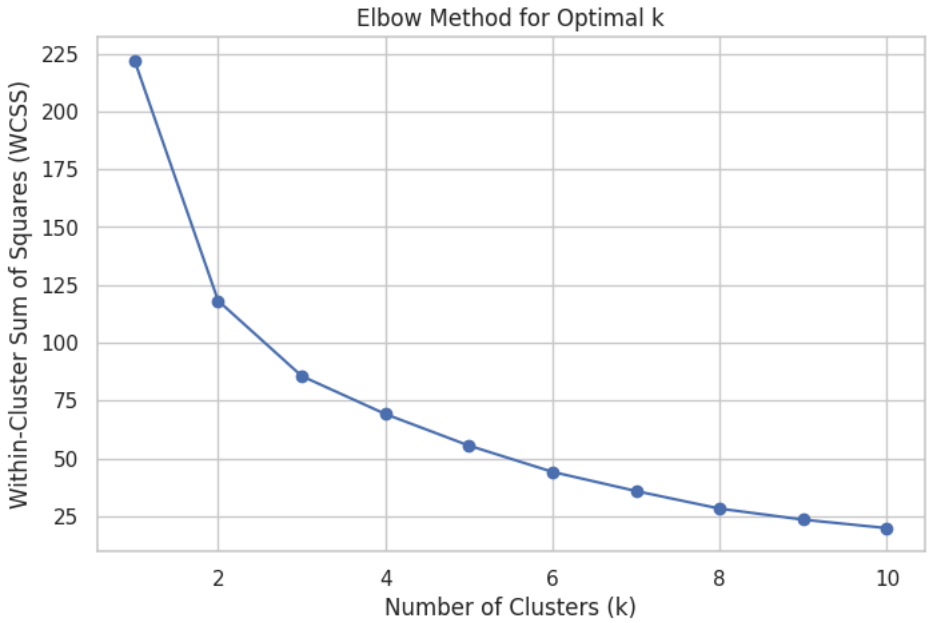
```
plt.ylabel('Scaled avg_critic')
plt.title('K-means Clusters')
plt.show()
```



8.950000	2	1
8.783333	6	1
8.762500	8	1
8.563636	11	1
8.484211	19	1
8.037500	8	1

```
#我们要用的333333333333
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

#Load data
data = pd.read_csv("/content/VideoGames (1).csv")

#Data Cleaning and Preparation
data = data.copy()
data['Critic_Score'] = pd.to_numeric(data['Critic_Score'], errors='coerce')
data['User_Score'] = pd.to_numeric(data['User_Score'], errors='coerce')

#Add Decade column
data['Decade'] = (data['Year_of_Release'] // 10) * 10

#Aggregate one row per (Genre, Decade)
gp_stats = (data.groupby(['Genre', 'Decade'])
              .agg(avg_NA     = ('NA_Sales',     'mean'),
                   avg_EU     = ('EU_Sales',     'mean'),
                   avg_JP     = ('JP_Sales',     'mean'),
                   avg_Other  = ('Other_Sales',  'mean'),
                   avg_critic = ('Critic_Score', 'mean'),
                   avg_user   = ('User_Score',   'mean'),
```

```
                  n_games    = ('Genre',        'count'))
              .reset_index())

#Feature matrix for clustering
X = gp_stats[['avg_NA', 'avg_EU', 'avg_JP', 'avg_Other', 'avg_critic', 'avg_user']]

#Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

#Elbow Method
wcss = []
k_range = range(1, 11)  # Test k from 1 to 10
for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=0, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)  # Inertia is WCSS

#Plot the Elbow Curve
plt.figure(figsize=(8, 5))
plt.plot(k_range, wcss, marker='o')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Within-Cluster Sum of Squares (WCSS)')
plt.grid(True)
plt.show()

#Select k based on elbow and run final clustering (e.g., k=3 as an example)
k = 3  # Replace with the k value from the elbow plot
kmeans = KMeans(n_clusters=k, random_state=0, n_init=10)
labels = kmeans.fit_predict(X_scaled)
gp_stats['cluster'] = labels

#Silhouette score
print("Silhouette score:", round(silhouette_score(X_scaled, labels), 3))

#Detailed view of clusters
print("\nGenre × Decade clusters:")
for c in sorted(gp_stats['cluster'].unique()):
    print(f"\n—— Cluster {c} ——")
    display(gp_stats.loc[gp_stats['cluster'] == c]
                    .sort_values('avg_user', ascending=False)
                    .reset_index(drop=True))

#Numeric summary of clusters
summary = (gp_stats.groupby('cluster')
                   .agg(pairs       = ('cluster',    'size'),
                        mean_NA     = ('avg_NA',     'mean'),
                        mean_EU     = ('avg_EU',     'mean'),
                        mean_JP     = ('avg_JP',     'mean'),
                        mean_Other  = ('avg_Other',  'mean'),
                        mean_cscore = ('avg_critic', 'mean'),
                        mean_uscore = ('avg_user',   'mean'))
                   .round(2))
print("\nCluster-level means:\n", summary)
```




```
Silhouette score: 0.471

Genre × Decade clusters:

— Cluster 0 —
```

	Genre	Decade	avg_NA	avg_EU	avg_JP	avg_Other	avg_critic	avg_user	n_games	cluster
0	Platform	1990	1.050000	0.578333	0.325000	0.098333	86.000000	8.783333	6	0
1	Role-Playing	1990	0.582632	0.438421	0.700526	0.100526	82.578947	8.484211	19	0
2	Shooter	1990	1.395000	0.292500	0.022500	0.037500	82.500000	8.425000	4	0
3	Simulation	1990	0.130000	0.087500	0.455000	0.040000	85.500000	8.175000	4	0
4	Misc	1990	0.150000	0.095000	0.820000	0.025000	79.000000	8.150000	2	0
5	Fighting	1990	0.822500	0.585000	0.530000	0.083750	79.125000	8.037500	8	0

```
— Cluster 1 —
```

	Genre	Decade	avg_NA	avg_EU	avg_JP	avg_Other	avg_critic	avg_user	n_games	cluster
0	Strategy	1990	0.016000	0.038000	0.000000	0.004000	81.600000	8.320000	5	1
1	Puzzle	1990	0.075000	0.050000	0.000000	0.010000	81.000000	7.850000	2	1
2	Role-Playing	2000	0.307582	0.142343	0.188759	0.050707	71.936869	7.746582	397	1
3	Adventure	1990	0.270000	0.125000	0.250000	0.025000	68.500000	7.550000	2	1

```
# Summary statistics
```